# Paging vs Segmentation

| Paging | Segmentation |
| --- | --- |
| Memory is divided into equal sized parts of page | Memory is divided into variable-sized segments based on logical parts of the program. |

ex. Imagine you have a word file with:

1. Title Page → 2 KB
2. Main content → 5 KB
3. References → 3 KB

→ Assume that memory divided into blocks of 4 KB each.

① Paging (Fixed size Blocks)

- Page 1 (4 KB) → Part of Title Page + Part of Main content
  2KB, 2KB
- Page 2 (4 KB) → Part of remaining main content (3 KB) + 1 KB of Referecens (1 KB)
- Page 3 (4 KB) → Remaining 2 KB of [illegible]

↳ 2 KB empty

↳ Internal Fragmentation

② Segmentation (Variable Size blocks)

- Segment 1 (2 KB) → Title Page
- Segment 2 (5 KB) → Main Content
- Segment 3 (3 KB) → References

- External fragmentation occurs as processes are loaded & removed from the memory, the free memory space is broken into little pieces.

Page Fault : When a running process tries to access a page but the requested page is currently not loaded into main memory then it is called page fault.

→ When a page fault occurs, the OS has to replace an existing page with the newly required one.
- The process of replacement is sometimes called swap out or write to disk.

→ To ~~minim~~ minimize the number of page faults, ~~page~~ page replacement algorithms are used.
i.e. FIFO, LRU → Least Recently Used
↓
First In First Out

# (40) Page Replacement Algorithm - FIFO

FIFO - First In First Out

→ This algorithm replaces the oldest page that has been present in the main memory for the longest time.

- whenever a page needs to be replaced, the first page in the queue is removed.

Advantages: It is simple and easy to use.

Disadvantages: The number of page faults increases by increasing the number of frames.

- Poor Performance.

Example: Consider page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7 with 3 page frames.

Find the number of page faults & hits.

Reference String :

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 1 | 1 | ~~1~~ | 0 | 0 | ~~0~~ | 3 | 3 | 3 | 2 | 2 |
| | 0 | 0 | ~~0~~ | 3 | 3 | ~~3~~ | 2 | 2 | ~~2~~ | 1 | 1 | 1 |
| 7 | 7 | 7 | 2 | 2 | ~~2~~ | 4 | 4 | ~~4~~ | 0 | 0 | 0 | 7 |

Total number of page faults = 13

hits = 5

$$\text{Hit Ratio} = \frac{\text{No of hits}}{\text{no of references}} \times 100 = \frac{5}{18} \times 100 = 27.77\%$$

$$\text{Fault Ratio} = \frac{\text{No of faults}}{\text{no of ref- stri.}} \times 100 = \frac{13}{18} \times 100 = 72.22\%$$

(A2)

# Page Replacement Algo. : LRU

## LRU - Least Recently Used

→ This algorithm replaces the page that has been used the least recently.
i.e. it replaces the page that has not been referenced for the longest time.

**Advantages :**

- more efficient than other replacement algorithms.

**Disadvantages :**

- expensive & complex to implement

**Example :** Consider page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7 with 3 page frames. find the no of page faults.

Reference string :

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7

| 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 | 0 | 1 | 7 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 1 | ~~1~~ | | 3 | | 3 | 2 | 2 | 2 | | | 2 | | 2 | | 7 |
| | 0 | 0 | 0 | | 0 | | 0 | 0 | 3 | 3 | | | 3 | | 0 | | 0 |
| 7 | 7 | ~~7~~ | 2 | | 2 | | 4 | 4 | 4 | 0 | | | 1 | | 1 | | 1 |

Page faults = 12 fault ratio = $\frac{12}{18} \times 100\%$ = $\frac{2}{3} \times 100\%$ = 66.66%

Page hits = 6 Page ratio = $6/18 \times 100\%$ = $1/3 \times 100\%$ = 33.33%